

LR(1) parser

Input

- 測資會放在" input" 資料夾內，資料夾中有3組公開測資



1_grammar.txt



1_testdata.txt

- 每組測資中有兩個檔案
 - x_grammar.txt：第1、2行為terminal及non terminal，接下來為Context-Free Grammar

- x_testdata.txt：多個syntax

```
terminal: a, b
nonterminal: S, A
S->AA
A->aA|b
```

```
bb
ab
abab
aaaaaabb
abaaab
abbbbbbb
abbababab
abaaab
abb
abaaaaaaaaaaaaaab
```

- 另有2組隱藏測資，格式與公開測資完全相同

- 轉換成Parsing table的過程

```
["S'->S", 'S->AA', 'A->aA', 'A->b']
////////// state //////////
0 : [[["S'->S.", ['$']], ['S->.AA', ['$']], ['A->.aA', ['a', 'b']], ['A->.b', ['a', 'b']]]
1 : [[["S'->S.", ['$']]]
2 : [[['S->A.A', ['$']], ['A->.aA', ['$']], ['A->.b', ['$']]]
3 : [[['A->a.A', ['a', 'b']], ['A->.aA', ['a', 'b']], ['A->.b', ['a', 'b']]]
4 : [[['A->b.', ['a', 'b']]]
5 : [[['S->AA.', ['$']]]
6 : [[['A->a.A', ['$']], ['A->.aA', ['$']], ['A->.b', ['$']]]
7 : [[['A->b.', ['$']]]
8 : [[['A->aA.', ['a', 'b']]]
9 : [[['A->aA.', ['$']]]
```

- Parsing table

```
////////// parsing table //////////
```

		\$	a	b	S	A
0			s3	s4	1	2
1	Acc					
2			s6	s7		5
3			s3	s4		8
4			r3	r3		
5	r1					
6			s6	s7		9
7	r3					
8			r2	r2		
9	r2					

Output

- 判斷每一筆syntax是否合法並列出過程

```
parsing: ab
result: Invalid!
```

stack	input	action
0	ab	s3
0a3	b\$	s4
0a3b4	\$	X

```
parsing: bb
result: Valid!
```

stack	input	action
0	bb	s4
0b4	b\$	r3
0A2	b\$	s7
0A2b7	\$	r3
0A2A5	\$	r1
0S1	\$	Acc

```
parsing: aca
Invalid character exist!
result: Invalid!
```

Grade

- 公開測資 40%
- 現場demo 40%
- 隱藏測資 20%
- 遲交：兩週內:此作業總成績*70% 、兩週後:0分
- 抄襲：0分

Others

- 繳交檔案
 - 原始碼：cpp、py等，請勿上傳整個專案
 - Readme.txt：編譯環境
 - 所有檔案壓縮成「學號_hw1.zip」或「學號_hw1.rar」
- 繳交期限：12/20 23:59
- Demo
 - 未demo者一律0分